Journal of Systems Architecture 137 (2023) 102848

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Enabling memory access isolation in real-time cloud systems using Intel's detection/regulation capabilities

Giorgio Farina [a,*], Gautam Gala [b], Marcello Cinque [a], Gerhard Fohler [b]

[a] Federico II, University of Naples, Naples, Italy
[b] Technische Universitaet Kaiserslautern, Germany

ARTICLE INFO

MSC:
0000
1111

Keywords:
Mixed-criticality systems
Real-time systems
Cloud computing
Industry 4.0
Intel

ABSTRACT

The increasing interest in adopting cloud technologies for Industry 4.0 and mixed-criticality environments is paving the way for compelling new opportunities and challenges. However, cloud deployments use multicore processors that introduce interference between co-executing applications on different cores due to contention in shared resources, such as the memory subsystem. Such interference can cause critical applications to miss their deadlines.

To enable the co-execution of critical and non-critical applications on the same multicore processor, we propose an approach that guarantees memory access time isolation for critical cores, while not jeopardizing the memory bandwidth of the non-critical ones. We prove the viability of our approach using Intel's resource director technology for memory access detection and regulation. Experiments show that queue occupancy is an excellent metric to estimate the number of interfering cores co-accessing the memory. We also assess the indirect memory bandwidth limitation achievable by applying Intel's Memory Bandwidth Allocation technology.

1. Introduction

Cloud computing presents the potential to improve efficiency and resource utilization, provide ease of reusability and reconfiguration, as well as higher scalability and availability together with the reduction in maintenance [1]. In addition, it allows industries to reduce their carbon footprint as a result of a reduction in resource over-provisioning. Safety-critical industries can also benefit from cloud computing as it opens a new market segment: safety-critical applications as cloud-based services. EU initiatives and projects such as Digitale Schiene Deutschland [2,3] and SECREDAS [4] have looked into cloud computing for hosting safety-relevant railway applications. Existing works (e.g. [5]) have also proposed virtualized railway signaling as a cloud-based service. TransVital [6] and DS3 [7] are up-comping platforms from Thales and Siemens to support safety-critical railway applications, such as interlocking and radio block center, in a SIL4 Cloud [8]. The automotive industry is moving towards software-defined vehicles and is exploring Cloud-native as the design paradigm [9]. The Scalable Open Architecture for Embedded Edge (SOAFEE) project [10] is an example of collaboration by automakers, semiconductor suppliers, open source and independent software vendors, and cloud technology leaders (e.g., ARM, AWS, Bosch, Cariad, Continental, and RedHat) to deliver a cloud-native architecture that aims to support mixed-criticality automotive applications. Eker et al. [11] from Ericsson highlight the need and requirements of real-time cloud computing in industry 4.0 use cases such as smart manufacturing.

In a cloud, applications can run on the same node and share the underlying node resources. Existing cloud virtualization environments, however, provide limited isolation amongst the applications [4]. Popular real-time hypervisors focus mainly on CPU virtualization, adopting either hierarchical scheduling [12] or CPU partitioning [13], and memory space virtualization among the applications. However, no allocation guarantees are given for other resources shared among applications. These include the Last Level Cache (LLC), the shared interconnect, and the memory bandwidth.

Memory access contention deserves special attention for two main reasons: (i) memory accesses are unavoidable [14], and (ii) memory access interference can increase the Worst Case Execution Time (WCET) by several orders of magnitude, contrarily to other sources of temporal contention [15], particularly when increasing the number of co-accessing cores [16,17]. The problem is exacerbated on modern multicore architectures for cloud computing, where many cores (or hardware threads) share additional resources, such as the LLC.

The LLC has a significant impact on the memory access contention, since (i) co-running cores can cause the eviction of critical cache lines,

* Corresponding author.
E-mail address: giorgio.farina@unina.it (G. Farina).

https://doi.org/10.1016/j.sysarc.2023.102848
Received 7 August 2022; Received in revised form 17 January 2023; Accepted 16 February 2023
Available online 20 February 2023
1383-7621/© 2023 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Acronyms

CAS	Column Address Strobe
CAT	Cache Allocation Technology
CDP	Code and Data Prioritization
CLOS	CLass Of Service
COTS	Commercial Off-The-Shelf
DRAM	Dynamic Random Access Memory
HP	Hyperthreading
iD&C	Incremental Development and Certification
IMC	Integrated Memory Controller
LLC	Last Level Cache
MBA	Memory Bandwidth Allocation
MPAM	Memory System Resource Partitioning and Monitoring
MSR	Model Specific Register
PCI	Peripheral Component Interconnect
PMC	Performance Monitoring Counters
PMON	Performance Monitoring
RDT	Resource Director Technology
RMID	Resource Monitoring Identifier
RPQ	Read Pending Queue
VM	Virtual Machine
WB	Write-Back
WCET	Worst Case Execution Time

making the synchronous memory bandwidth consumed from critical tasks unpredictable or severely pessimistic; (ii) memory writes cannot be associated with physical cores. Several works [18,19] consolidated space partitioning techniques for the LLC, even when the mapping address function is more complicated than a module [20]. In addition, recent works explored the Intel hardware-based Cache Allocation Technology (CAT) [21–23].

Once the partitioning of the LLC is implemented, memory bandwidth regulation is still necessary to limit memory access contention. On this line, resource limitation approaches can be used to find tight upper bounds for the memory fetching phase of a critical task running on a core. This is done by limiting the memory accesses of non-critical tasks running on other cores, within a regulation period. According to the Incremental Development and Certification (iD&C)[24], the new guaranteed minimum memory bandwidth is estimated by reproducing the limited maximum interference. Hence, memory access isolation is guaranteed for the critical task by reaching a new minimum memory bandwidth.

The most popular software-based implementation is [25]. It monitors the memory requests (or cache misses) from each core. It applies a throttling policy to prevent memory accesses from cores that have depleted the assigned memory budget for that regulation period. However, the throttling policy of the approach involves stalling the core execution to prevent memory access. Such a throttling policy is unsuitable for cloud Virtual Machine (VM) as a core cannot continue its execution within the private context, such as private caches and functional units, reducing the private resources utilization.

Luckily, modern processor manufacturers introduced support for hardware-level resource partitioning [26]. In our previous work [27], we assessed the indirect memory bandwidth limitation of Intel MBA available on the 2nd generation Xeon Processors. It is a hardware controller to limit the memory bandwidth of non-critical cores. In particular, MBA delays the requests going to the High-speed interconnect from a core's private context, as depicted in Fig. 1. Intel provides nine values between 10 to 90 in increments of 10, representative of

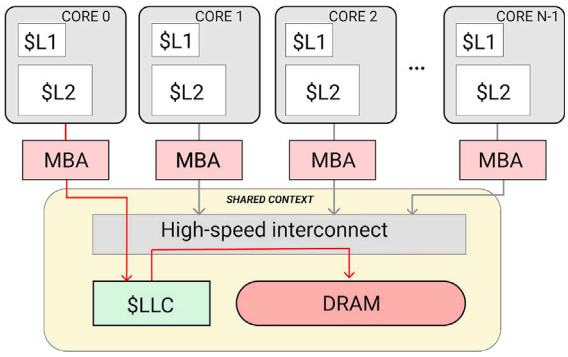


Fig. 1. Abstract model of Intel Xeon memory hierarchy depicting the MBA controller between the cores and high-speed interconnect.

the delays inserted between the requests. The memory bandwidth allocation/regulation is an indirect consequence of the introduced delay; therefore, it needs a systematic evaluation. Nevertheless, once we have a way to enforce static memory bandwidth partitioning, imposing the regulation even when it is not necessary can reduce the overall memory bandwidth utilization. Therefore, in this paper, we extend our previous work in [27] designing a solution to enable the static partitioning only when the interference (in terms of co-running cores) can impact the guarantees of critical tasks.

To support such a solution, we exploit and assess a new monitoring feature of Intel processors to detect the current degree of interference. Hence, the main contributions and results of the study are as follows:

1. We design a predictable detection/regulation approach to improve the achievable memory bandwidth of non-critical cores while providing isolation to the critical one.
2. We prove experimentally that the queue occupancy enables the detection of the number of co-accessing cores showing that the detection time is at most one-fifth of the regulation period in the worst case when the memory throughput is fully consumed.
3. We derive the indirect memory bandwidth limitation of Intel MBA from Intel delay values using a single-core analysis, where we highlight the importance of considering a worst-case workload that we name "Exclusive DRAM Bomb".
4. We test the derived assured memory fetching latency for a core allocated to a critical VM when MBA limits the bandwidth of three interfering cores with a multicore analysis, showing the potential and effectiveness of the approach.

The rest of the paper is structured as follows. Section 2 presents the related work, while the design of the detection and regulation approach to improve memory bandwidth utilization is presented in Section 3. The background on the new Intel features for monitoring and regulation which make feasible the approach is provided in Section 4. Subsequently, Section 5 introduces the shared parameters of the experiments, Section 6 assesses the detection support and Section 7 and Section 8 assess the regulation features. Finally, Section 9 ends the paper with final remarks and hints for future work.

This paper significantly extends our previous contribution in [27] by introducing the new predictable detection/regulation approach in Section 3, along with its experimental evaluation in terms of queue occupancy in Section 6. In addition, we greatly extended the related work and background, respectively in Section 2 and Section 4.

2. Related work

In multicore processors, single task worst-case CPU utilization/execution time (WCET) estimation is highly pessimistic, especially, without partitioning the memory bandwidth [17]. Some approaches

develop completely new specialized real-time processor architecture designs to tackle such issues. For example, Edwards and Lee [28] suggested the Precision Timed (PRET) architecture to deliver timing predictability and repeatability. Reineke et al. [29] proposed a new Dynamic Random Access Memory (DRAM) controller design for the PRET architecture to improve timing predictability and isolation between tasks at DRAM level. Contrarily, this paper focuses on Commercial Off-The-Shelf (COTS) multicore processors, and thus, making changes to existing hardware architecture is not applicable here.

Existing work proposes some methodologies to analyze the impact of shared resources (including memory bandwidth) on the WCET of tasks in COTS multicore processors. For example, Pellizzoni et al. [17] show how to upper bound the task delay due to memory contention. Chattopadhyay et al. [30] present a unified WCET that considers shared caches, on-chip interconnect, and other micro-architectural components such as instruction pipeline and branch predictor. Such methods involve searching vast state spaces to include all possible task interactions. As a result, they suffer from enormous computational complexity.

Interference regulation approach types in existing literature can be broadly divided into the following types: Execution Model, Resource Limitation, and Resource Reservation. Some works also explore the combination of some of these approaches. Intel MBA [31] is a hardware-based solution of the Resource Limitation approach. Below, we review these main approaches.

2.1. Execution model

In [32], authors propose decomposing tasks into a fixed sequence of superblocks consisting of three phases: acquisition, execution, and replication. Memory accesses are only possible in the acquisition and replication phases, and the acquisition or replication phases of simultaneously executing tasks cannot overlap to avoid resource contention. This model requires tight static analysis, and minor design changes can significantly increase a task's worst-case response time.

Houdek et al. [33] provided building blocks for implementing the Predictable Execution Model (PREM) [34] on an ARM-based multicore processor. PREM splits task jobs into two non-preemptible intervals: 1. Predictable interval with no system calls and interrupts; 2. Compatible interval with no special provisions. The predictable interval has two phases: a memory phase to prefetch all task data and instructions into private caches and an execution phase to perform the required task computation predictably without resource contention. Tasks need code instrumentation to mark task sections for execution in predictable intervals and a specially designed compiler to use PREM model. Yao et al. [35] introduced a memory-centric scheduler to improve performance in a TDMA-based memory arbitration and considered a PREM model. When the TDMA scheduler grants a memory slot to a core, the memory-centric scheduler reduces the priority for all active jobs in the execution phase on this core as compared to those in the memory phase on the same core.

The sliced execution model [36] divides the CPU execution time into multiple slices of two types: 1. Execution slice: a core executes a task based on prefetched instruction and data; 2. Communication slice: the core flushes all data from the previous execution slice to the DRAM and prefetches the instruction and data required for the next execution slice. A special toolset is required to generate the sliced architecture.

Biondi et al. [37] use the Logical Execution Time model [38] that decouples a task's CPU execution phase and the communication phase. The authors restrict the memory accesses of each task (prefetching and writing to memory) to precise time windows (communication phase) located at the beginning of a task's period, thus avoiding memory contention by design. As noted by the authors, such a model can have priority inversion as the LET communication phase for a low-priority task can delay the execution of a high-priority task.

Such models often poorly utilize the resources [39]. Execution models in general are too invasive for Mixed-Criticality Systems. They require code or compiler modifications that make it challenging to transition single-core legacy applications to COTS multicore processors and can increase the re-certification costs.

2.2. Resource limitation

Approaches based on resource limitation free the non-critical tasks to be structured as execution models. They limit the number of accesses within a regulation period for non-critical tasks without modification to the original program. The limitation enables Incremental Development and certification [24] to define and reproduce the limited maximum interference for the memory fetching phase (the Acquisition phase) of the critical tasks. These solutions are usually based on a throttling policy to stop the processing element access in case of excessive requests and a monitoring policy for counting the issued requests. Notably, Bellosa [40] is among the first to perceive the potential of hardware performance counters to monitor shared resource accesses. The idea is to capture only the interesting available events on each processing element (PE), such as last-level cache hits and references, to extract the number of LLC misses (representing synchronous accesses). Subsequently, LLC misses are supported as non-architectural events on modern COTS processors and are also used to monitor PE accesses [41–44]. As a throttling policy, Bellosa [45] inserts idle loops inside the TLB-miss handler. Yun et al. [41] analytically calculate memory throttling parameters to guarantee memory bandwidth to a safety-critical task executing on one core while limiting the impact on non-critical tasks executing on all the other cores. They count LLC miss performance counter events to determine the number of memory accesses by a core. Nowotsch et al. [46] proposed the concept of interference sensitive WCET (isWCET), to account for shared resource contention in COTS multicore processors. isWCET is calculated offline by analyzing the WCET and tasks' resource usage in isolation and the computed shared resource interference delays from co-executing tasks. In addition, they have an online mechanism based on hardware performance monitor events to bound a task's (core's) maximum resource contention.

Few previous works adopt an existing hardware feature, Intel CAT [22], to enable the partitioning of the LLC, while some others [47–49] design bandwidth limitation feature at the hardware level. Pagani et at. [48,50] and Aghilinasab et al. [51] discuss the integration of application cores with accelerators and GPUs in a shared memory system, respectively. Serrano-Cases et al. [52] performed qualitative and quantitative analysis on various QoS-enabled IP blocks on a Zynq UltraScale+ and provide suggestions to control contention in the shared bus and memory controller at runtime. Envelope-aWare Predictive model (E-WarP) [50] is a framework to profile tasks executing on CPU cores and accelerators. The framework bounds the execution time of tasks by monitoring the memory controller activity via performance monitor events and enforcing memory-bandwidth regulation by a combination of Memguard [41] (for CPUs) and ARM QoS Support (for accelerators).

Recently, Intel also supported the Resource Limitation approach on the new Xeon Scalable Processors at the hardware level. Intel MBA delays the requests going to the High-speed interconnect from a core's private context, as depicted in Fig. 1. Intel provides nine values between 10 to 90 in increments of 10 representative of the delays inserted between the requests. The memory bandwidth allocation/regulation is an indirect consequence of the introduced delay. Similarly, Memory System Resource Partitioning and Monitoring (MPAM) [53] from ARM provide memory-access regulation mechanisms in A-profile ARM processors. Zini et al. [54] take a closer look at this technology and provide detailed instructions for MPAM instantiation. Recently, another work [21] has explored the indirect memory bandwidth limitation of the MBA. However, they adopt old synthetic benchmarks for evaluation. These benchmarks are insufficient to reproduce the worst case in the new micro-architectures. Contrarily, in our work [27], we build a

new synthetic benchmark with a new traffic pattern for the exclusive cache architecture, reproducing the worst-case interference. Taking inspiration from [21], we call this type of workload "Exclusive DRAM Bomb", considering the Exclusivity of the LLC.

2.3. Resource reservation

The last relevant approach is Resource Reservation. Memory banks can work in parallel; assigning distinct banks to critical and non-critical applications reduces contention and access variability. Several works [18,55–57] consolidate this solution with concrete implementations. However, the throughput is reduced in favor of predictability. Moreover, the binding of memory banks with physical addresses is generally not directly available on COTS platforms and requires reverse engineering techniques to be operational. PALLOC [58] extends Linux virtual memory system to allocate memory pages of tasks to specific DRAM banks.

Specialized virtualization technologies (e.g., XtratuM hypervisor [59]) provide strong spatial isolation at DRAM level by allowing a system designer to allocate memory regions to specific virtual machines. However, they do not support memory bandwidth allocation to VMs.

Yun et al. extended the resource limitation method [41] to develop MemGuard [25], a memory reservation mechanism that statically partitions memory bandwidth between cores. Memguard divides the memory bandwidth between guaranteed and best effort. Once all cores have exhausted their guaranteed bandwidth, they compete with other cores for the best-effort bandwidth. Mancuso et al. [60] propose a Single Core Equivalence (SCE) framework by statically allocating an equal amount of each shared resource, such as DRAM banks, memory bandwidth, and shared cache, to each core of the COTS multicore platform. The framework aims to provide a parametric WCET estimation for a task running on top of such a statically partitioned platform. Contrary to the static memory bandwidth allocation in the SCE framework, Agrawal et al. [61] consider dynamic memory bandwidth allocation and analyze the worst-case response time of a task executing in a sequence of intervals with different memory bandwidth allocations.

Behnam et al. [62] proposed a multi-resource server-based approach on the Freescale P4080 processor to provide guaranteed CPU bandwidth and memory bandwidth to tasks. For a slot-based time-triggered system executing on a COTS multicore, Agrawal et al. [63] proposed two servers per core and used the isWCET concept [46]. One server controls the processor time, and the other manages the memory bandwidth. Thus, both servers jointly control the contention between cores and the memory accesses per slot.

In DNA [23], the authors built an execution profile of tasks consisting of phases by analyzing their resource usage patterns. A runtime mechanism dynamically allocates resources to tasks based on tasks' phases. However, DNA only considers simple phase analysis and leaves considering complex task behavior in phase analysis to future work.

3. Our approach: Detection and regulation of the memory access interference

The problem we aim to solve is to make the maximum memory access interference of critical cores predictable. In other terms, we aim to provide memory access isolation to critical cores, while not compromising the functioning of non-critical cores (e.g., avoid throttling). Let us assume that it is possible to assign memory bandwidth to cores (feasible on modern Intel processors with MBA, as discussed in the next section). In the static partitioning example depicted in Fig. 2, the non-critical cores have a limitation that reduces their achievable memory bandwidth to 10%, 20%, 30%, and 40%, respectively, while the critical cores are allowed to freely access the memory. Such an approach is optimal for the critical cores; however, this approach reduces the average available memory bandwidth for non-critical, even when unnecessary. By detecting and regulating the interference, we aim to improve the memory bandwidth utilization of non-critical cores, that can still run performance-sensitive applications in a cloud setting.

The degree of interference can be inferred from the number of co-accessing cores. Via static partitioning, we can derive a threshold for the interference under which the WCET of critical tasks is guaranteed and then apply the regulation once the threshold is passed. For instance, the worst interference for a critical core (running a critical task) occurs when there are two additional interfering cores in Fig. 2; therefore, we could set a threshold to three cores (including the critical core). Hence, no regulation is needed until three cores are executing memory-intensive workloads within a memory bandwidth regulation period. At the same time, once the threshold is passed, memory bandwidth partitioning must be applied to reduce interference and not impact the critical core.

In the following sections, we consider a $2^n d$ Generation Intel Xeon Scalable Processor as the target architecture and explore the following research questions:

- **RQ1** Can the variation of the memory queue occupancy reflect the number of cores co-accessing the memory?
- **RQ2** Is it possible to estimate the time required to detect when more than k cores are co-accessing the memory?
- **RQ3** Is it possible to limit the memory bandwidth of the non-critical cores using Intel's MBA technology?
- **RQ4** Is it possible to guarantee the memory bandwidth of the critical core using Intel's MBA technology?

RQ1 is supported in theory. The number of pending requests for each physical core in the shared context is limited; hence, we expect a different number of co-accessing cores to achieve a distinct occupancy value. We call "detection time" the time window required in **RQ2**. It is a portion of the memory bandwidth regulation period where no bandwidth regulation is enforced. Hence, the tighter the detection time, the lower the overhead dedicated to the detection. For **RQ3** and **RQ4**, we need first to analyze which workloads affect the memory bandwidth limitation. MBA delays the requests going to the LLC. Hence, we included different traffic patterns targeting both, the LLC and the DRAM, in the analysis.

4. Overview of the intel monitoring/regulation capabilities

This section introduces Intel's recent hardware features that support the monitoring and regulation of events in the memory hierarchy.

4.1. Per-core Performance Monitoring Counters (PMC)

The per-core Performance Monitoring Counters (PMC)[1] count the performance monitoring events from each core's perspective. Often, each physical core has eight PMCs. In the case of Hyperthreading (HP), these are divided between the virtual cores hosted in the physical core. For instance, one core can configure one PMC to count the "LLC miss" events. Thus, the core tracks the number of LLC misses caused by itself.[2] The counter can be configured to generate an interrupt once it has reached a given number of events.

4.2. Un-core Performance Monitoring Counters (PMC)

The un-core PMC are local counters of the shared resources. They are generally unaware of the target core and readable from any core.

[1] Note Intel refers to the Per-core and Un-core Performance Monitoring Counters (PMC) as "PMC" and "un-core PMC" respectively.

[2] The "LLC miss" event is used by state-of-the-art approaches such as Memguard [25].

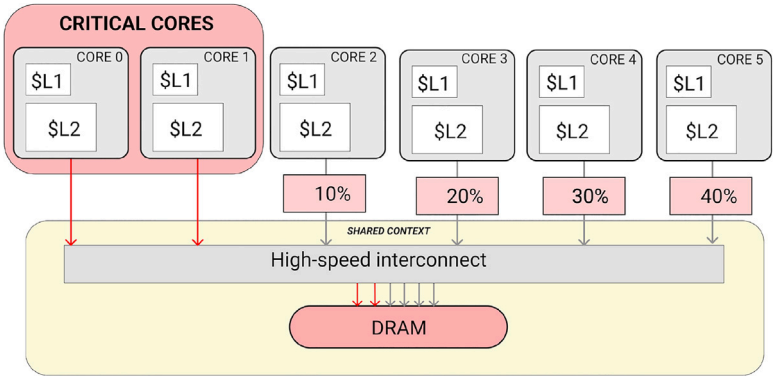


Fig. 2. Example of static memory bandwidth partitioning.

The un-core architecture has several boxes enclosing single physical components such as the high-speed interconnect, the memory controller, and the shared LLC. These boxes include Performance Monitoring (PMON) local blocks. Each PMON local block includes the control, status, and data for the counters. The programming of PMON local blocks is through Model Specific Register (MSR) or Peripheral Component Interconnect (PCI) interface. In addition, un-core monitoring includes the PMON global block, which (i) allows taking some actions on the PMON local blocks through a unique global structure. Furthermore, (ii) once the counters overflow, the PMON global block also allows interrupting a specified subset of physical cores to run a handler. In our case, once we pass the threshold on the RPQ occupancy, we could run the regulation routine. This feature can be adopted from previous works to make feasible the detection phase of our detection/regulation interference approach. Molka et al. [64] discuss the accuracy of un-core monitoring in detecting memory bandwidth utilization. Un-core monitoring is adopted both in real-time and security systems for reverse engineering [65,66] or in green computing to estimate and reduce the power consumption [67,68].

4.2.1. IMC boxes in our architecture

The Intel Xeon Scalable Processor un-core monitoring [69] has two memory controllers, which are respectively enclosed in two distinct IMC boxes. Each IMC box includes respectively one PMON local block for each DRAM channel. The IMC boxes are exposed through the PCI interface. One PMON local block is composed of four counters and one fixed counter. The fixed counter only tracks the number of DRAM Clocks (DCLK), which never changes frequency; therefore, it is a good measure of the wall clock. Instead, the other counters can monitor several events, such as DRAM refresh cycles, DRAM Column Address Strobe (CAS) commands, and the occupancy of one DRAM channel. We record the CAS commands in our experiments of Section 7. Memory writes (CAS write commands), in particular, can only be monitored globally[3] when the LLC is shared because other cores can also cause WBs.

4.2.2. The Read Pending Queue (RPQ) occupancy event

Read Pending Queue (RPQ) occupancy can be monitored via the IMC PMON box un-core monitoring in the Intel Xeon Scalable processor [69]. The RPQ is used to schedule reads out to the memory controller and to track the requests. Requests are allocated into the RPQ soon after they enter the memory controller, and need credits for an entry in this buffer before being sent to the IMC. The RPQ occupancy event accumulates the occupancy of the RPQ each DRAM clock[4] (see Section 2.3.7 of [69]). In particular, the setup includes the definition of a threshold on the occupancy; the counter counts the number of cycles that the occupancy (the number of queued requests) is higher than this threshold. For instance, if we set the threshold to one, the counter is increased every time the queue holds at least one request during one cycle. One entry is deallocated from the RPQ once the memory operation has been issued to the memory; hence, by setting the threshold to one, this counter provides the busy time (in cycles) of the queue.

4.3. Resource Director Technology (RDT)

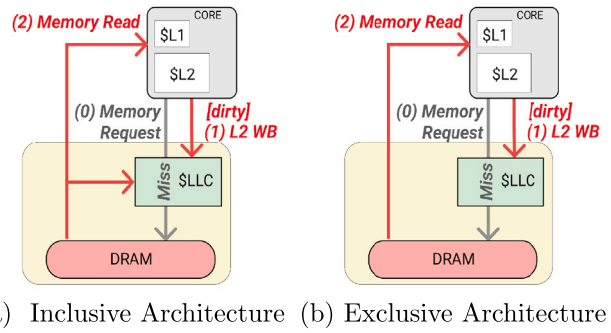
The 2nd Generation of Intel Xeon Scalable Processors [26] is based on the Intel Cascade-Lake micro-architecture. The considered micro-architecture and all the new generations of Intel Xeon Scalable processors support temporal and spatial isolation at the hardware level through Resource Director Technology (RDT) [31]. The Model Specific Register (MSR) allows interfacing with RDT. RDT programming includes associating spatial and temporal properties with a CLass Of Service (CLOS) and binding the CLOS to a virtual core. At the same time, RDT programming allows monitoring of the spatial and temporal consumption of shared resources by binding a Resource Monitoring Identifier (RMID) to a virtual core. Hence, each virtual core can be associated with CLOS and an RMID. One virtual core cannot change the class of service (or RMID) of another core. However, any virtual core can configure the properties of any CLOS and can read the monitored performance associated with an RMID. Using RMID, it is also possible to monitor the cache space occupancy and the memory bandwidth. They can be read via polling (i.e., in a synchronous way). No thresholds can be set to enable asynchronous handling. The RDT allocation features associated with CLOS enable the partitioning of shared resources between the cores. While Cache Allocation Technology (CAT) and Code and Data Prioritization (CDP) focus on the LLC spatial partitioning, Memory Bandwidth Allocation (MBA) addresses the temporal memory contention.

4.3.1. Memory Bandwidth Allocation (MBA)

MBA inserts a configurable delay between requests going to the high-speed Interconnect from a core's private context. The granularity of MBA control is linear, starting at 10% and going up to 100% in increments of 10%. Through the delays, we can apply an indirect limitation in memory access without impacting the use of private resources such as private caches and functional units. An MBA delay value is associated with a CLass Of Service (CLOS). This binding can be done in any core. For instance, we can associate the critical and non-critical cores with

[3] For instance, the LLC misses can be monitored as a per-core event because the LLC miss is caused by the physical core that issued the request. Instead, the LLC Write-Back (WB) can be caused by other physical cores that demand other data, and thereby it is not provided as per-core event.

[4] One DRAM clock corresponds to half of the DRAM Speed



(a) Inclusive Architecture (b) Exclusive Architecture

Fig. 3. Last Level Cache miss handling.

distinct classes of service, $CLOS_X$, and $CLOS_Y$, respectively. Once the critical cores access the memory, we can limit the memory accesses of non-critical cores, changing the delay of $CLOS_Y$.

Note two virtual cores share one physical core. If two virtual cores (hardware threads) of the same physical core have different delays, the larger delay will override the smaller one. In our micro-architecture, disabling Hyperthreading (HP) in the experiments, we have a side effect [26]; in particular, the non-active hardware threads are assigned to the CLOS0 by default even if they are disabled. Hence, CLOS0 cannot be used. Otherwise, the hidden delay value of the non-active virtual core (the delay value associated with the CLOS0) overrides the set MBA delay value of the active virtual cores on the same physical core when the first delay is higher than the second one.

4.3.2. Exclusive Last Level Cache (LLC)

The Cascade-Lake and the newer Ice-Lake micro-architecture inherited from the Skylake-Server micro-architecture have an exclusive LLC and a mesh network as a shared interconnect. The property of exclusivity changes the memory traffic patterns. The design of experiments in Section 7 and Section 8 considers the exclusive traffic patterns. Fig. 3 shows how the inclusive and exclusive architectures handle the LLC misses, respectively. Whenever an LLC miss occurs in inclusive architecture, the new cache line fills both LLC and private cache, while, in exclusive architecture, only the private cache is replenished. The two figures suppose that there is available space in LLC. The different reactions to the LLC miss influence memory read workloads. Suppose we start with empty private caches. Once we fill up all the private caches, we do not have an additional L2 Write-Back (WB) in the inclusive architecture because the cache lines are already updated and available in LLC. In the exclusive architecture, instead, there are L2 WBs. Unlike previous works (e.g., [21]), we consider this behavior while evaluating MBA to obtain more accurate results regarding the worst-case interference.

4.4. Summary

In this section, we provide a summary of the presented hardware features that support temporal and spatial isolation in the memory hierarchy.

Table 1 describes an *Action* by using a given *Technology*. The first half of the table includes monitoring capabilities, while the second half includes regulation capabilities. The *Action* is taken from a core and *targets* a set of cores (*Multi-core*), or the core where the action is taken (*Per-core*), or any cores (*Un-core*, meaning "unaware of the core"). The programming hardware interface is specified in *Interface* (i.e., by MSR or PCI interface). The column *Int.* specifies if the monitoring feature supports the generation of asynchronous events once a threshold is specified. All these *Actions* can be taken online.

For further clarity, we explain the first two rows of the table. In the first row, the core sets the RMID targeting itself (*Per-core*) with a RMID. While in the second row, the core reads the monitored performance associated with a RMID. In this case, it reads the monitored performance of all cores that are targeted with that RMID (*Multi-core*). It is not possible to set a threshold on the monitored events to cause an interruption.

5. Overview of experiments

Table 2 describes the shared parameters of the experiments. We use only one socket and set the number of active cores via BIOS configuration. We used only one memory channel in both analyses. We set the DRAM refresh rate to periodic and the CPU core frequency to maximum (no power saving) in the BIOS. These parameters help keep the minimum latency variability and maximize the memory requests rate. Similarly, we disabled the prefetcher and hyper-threading to reduce variability since hardware threads on the same physical core share the same "queue" to the memory and the prefetcher can preload several lines after a single cache miss. Experiments are conducted on an Intel Xeon Silver processor (Cascade-Lake micro-architecture) with eight cores. Each core runs at 2.1 GHz and has a 512 KB L1 cache (256 KB out of 512 KB are dedicated to instructions), and a 1MB L2 cache. All cores share an 11MB LLC.

In the experiments, our measures achieve the 99% accuracy calculated as the 99% mean confidence interval using a student's t distribution divided by the sample mean.

6. Assessing detection support: RPQ occupancy

This section investigates if the queue occupancy works as a metric to detect the number of co-accessing cores and the time required to detect when more than k cores are co-accessing the memory (*detection time*), i.e., **RQ1** and **RQ2**.

6.1. Experiment setup

Table 2 and Table 3 include the parameters and factors considered in the analysis.

Metric. Our experiments measure the portion of time $P_{i,j}$ within a regulation period of one millisecond in which the RPQ occupancy[5] is greater than or equal to a given value i of requests when j cores contend the memory access. In particular, the monitoring counter requires setting the threshold i on the RPQ occupancy. The counter is incremented if the RPQ occupancy is at least i during a DRAM clock cycle. Hence, once we reproduce memory interference from j co-accessing cores to the memory, the monitoring counter provides the number of cycles $Cycles_{i,j}$ in which the RPQ occupancy was at least i. $P_{i,j}$ is obtained by normalizing $Cycles_{i,j}$.[6]

Factors. We change the number of co-accessing physical cores j from one to six, where every core reproduces only memory read operations. We set the RPQ occupancy threshold i in a multiple of eight being enough to describe and include the total response variation.

Parameters. We set the regulation period to one millisecond similar to Memguard [25]. The workload is composed of continuous memory read operations. They are generated by allocating and reading a dynamic memory bigger than the LLC size. Thus, for each memory read request, one memory read and one L2 WB[7] are generated. The additional L2 WB does not significantly affect the arrival rate, while it helps to have a continuous flow of memory read operations within the regulation period.

[5] The queue occupancy is the number of queued requests at a given instant of time.

[6] The monitoring counter can measure at maximum the regulation period (i.e., one millisecond) in terms of DRAM clock cycles.

[7] The L2 WB is generated due to the exclusive LLC.

Table 1
Summary of Intel's monitoring and regulation technologies.

Technologies	Properties			
	Action	Interface	Targeting	Int.
RDT	setting RMID	MSR	Per-core	no
CMT	reading RMID perf.	MSR	Multi-core	no
MBM	reading RMID perf.	MSR	Multi-core	no
Per-core PMC	reading PMC	MSR	Per-core	yes
Un-core PMC	reading PMC	MSR/ PCI	Un-core	yes
RDT	setting CLOS	MSR	Per-core	–
CAT	ways to CLOS	MSR	Multi-core	–
MBA	delay to CLOS	MSR	Multi-core	–

Table 2
Design of experiments: shared parameters.

Shared parameters	Setting
No. sockets	1
Used DRAM channel	1
Prefetcher	disabled
SMT	disabled
DRAM refresh cycles	periodic
Core frequency	maximum

Table 3
Design of experiments: RPQ occupancy.

Metric	Description
RPQ occupancy cycles	$P_{i,j} = \frac{Cycles_{i,j}}{maxCycles}$
Factors	Values
RPQ occupancy thresholds	$i \in T, T = \{1; 8; 16; 24; 40; 47\}$
No. co-accessing cores	$j \in C, C = \{1; 2; 3; 4; 5; 6\}$
Parameters	Setting
Regulation period	1 ms
Workload	memory read operations

6.2. Experiment observations

Fig. 4 shows the percentage of the regulation period in which the RPQ occupancy is greater than or equal to i requests when j cores are co-accessing the memory. Fig. 4 (a) plots the percentage of time in which the queue holds at least one request representing the time in which the DRAM Channel is busy to service read requests.

6.3. Discussion

When two or more cores contend for memory access, the queue is busy servicing requests (Fig. 4 (a)) for almost the whole regulation period. In such a condition, the memory controller services requests continuously and the memory throughput reaches its maximum (around 280,000 memory read operations in one millisecond). Hence, once exceeded by the two co-accessing cores, the throughput cannot represent the number of co-accessing cores. At most, it can only indicate when there are at least two cores contending to access memory once half of the maximum throughput is consumed in half of the regulation period (detection time). However, while the throughput is constant once it is fully consumed, the queue occupancy grows. We expect that the queue occupancy does not grow indefinitely because the pending requests of the physical cores are limited. The other plots of Fig. 4 confirm this assumption. From the results shown in Fig. 4, we can observe that the queue occupancy reflects the number of the co-accessing core to the memory responding to the **RQ1**. In particular, visibly, the occupancy varies when more than 2, 3, 4, and 5 cores are co-accessing the memory in Fig. 4 (b), (c), (d), (e). For instance, setting j to two and three (i.e., two and three co-accessing cores), RPQ holds at least eight requests for one-fifth or for the entire regulation period ($j = 2$ and $j = 3$ of Fig. 4 (b)).

Thus, we adopt occupancy as a metric for detecting the co-accessing cores. We define the detection time, DT, as the portion of the regulation period dedicated to the detection of occupancy. It can be calculated as follows (responding to **RQ2**):

$$DT_{i,k} = (1 - P_{i,k+1}) + P_{i,k}$$

where i is the RPQ occupancy threshold fixed on the queued requests and k is the number of co-accessing cores.

For the sake of simplicity, we explain this formula through an example where we want to detect more than two co-accessing cores ($k = 2$) by monitoring the cycles in which the queue at least holds eight requests ($i = 8$). Observing Fig. 4 (b), when there are three physical cores, the queue holds at least eight requests for almost all of the regulation period. However, there is a slight difference ($maxCycles - Cycles_{8,3}$) because the ideal maximum throughput is still not fully achieved. Since we do not know the occupancy distribution in the regulation period, we need to assume the worst case.

In the worst case, after at most $(1 - P_{8,3})$ of the regulation period, the queue starts to hold at least eight requests when at least three co-accessing cores are active. Hence, under three co-accessing cores, the monitoring counter exceeds the cycles $Cycles_{8,2}$ reached from two-co accessing cores after almost $DT_{8,2} = (1 - P_{8,3}) + P_{8,2}$ of the regulation period. Hence, by dedicating $DT_{8,2}$ of the regulation period in monitoring, we can detect if more than two cores are contending the memory. From a practical point of view (see Section 4), we can set an interrupt once the monitoring counter overflowed $Cycles_{8,2}$.

The second column of Table 4 shows the minimum detection time $min(DT_{occ})$ to detect more than k co-accessing physical cores to the DRAM channel by observing the variation of the occupancy. The third column provides the occupancy threshold tuned to reach the minimum detection time, while the fourth shows the detection time if we have adopted the variation of the throughput as the observable metric. If the detection time is one, the detection requires the entire regulation period to become unfeasible. The results are promising: at most, one-fifth of the regulation period is enough for detection when the total memory bandwidth is consumed. In particular, only detecting more than two cores $k = 2$ requires one-fifth of the regulation period while detecting more than three, four, and five cores $k = 3, 4, 5$ requires one-sixth. Ideally and theoretically, we expect a detection time equal to zero when the memory throughput is fully consumed, e.g., we presume that some queue positions are never achievable when the number of the co-accessing core is limited. However, the ideal case does not include real sources of interference such as the DRAM Refresh Cycles in which the memory stops to service requests.

Therefore, the occupancy reflects the number of co-accessing cores to the memory, even if the detection time can be further improved by locking other factors of interference such as the refreshes of the DRAM.

7. Assessing regulation support: Single-core analysis

The single-core analysis estimates the indirect memory access limitation for an interfering core when MBA is enabled with different delays, i.e., **RQ3**.

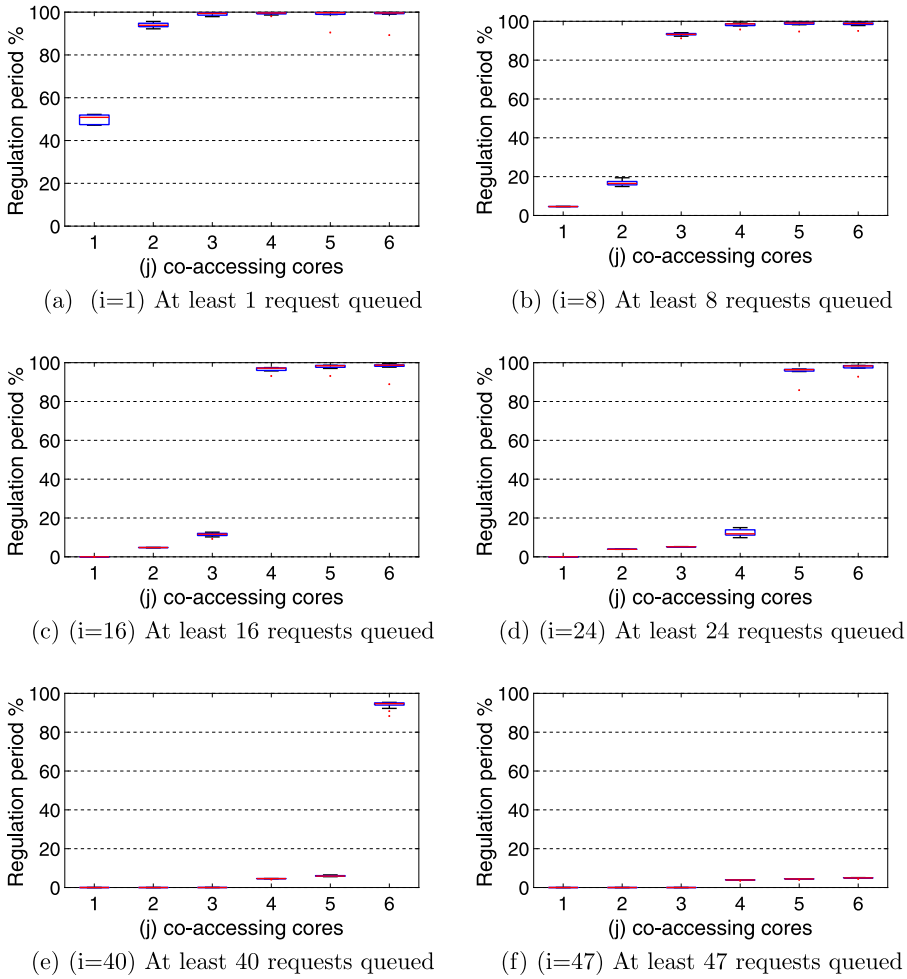


Fig. 4. Portion of the regulation period in which the RPQ occupancy is at least i when j cores are co-accessing the memory.

Table 4
Minimum detection time to detect more than k co-accessing cores.

k	$min(DT_{occ})$	$Threshold_{occ}$	$min(DT_{tp})$
1	0.56	1 reqs	0.50
2	0.23	8 reqs	0.94
3	0.15	16 reqs	1
4	0.16	24 reqs	1
5	0.11	40 reqs	1

7.1. Experiment setup

Table 5 describes the design of experiments for the single-core analysis.

Metrics. Our experiments measure the execution latency of k workloads. $L_{k;d}$ is the execution latency of the workload k when executed with MBA set to a delay value d.

Factors. Intel provides nine possible delay values between 10 to 90 with a stride of ten. These values have a linear effect in delaying the requests going to the LLC. No additional information is provided from Intel. We exclude 20 and 30 delays from the analysis as they are erroneous on our processor model. As per the errata [26] provided by Intel, these two delay values produce the same results as the 10 delay value. We consider three workloads in our experiments covering all the possible LLC traffic patterns. The memory reads workload considers only synchronous memory accesses, while the memory writes workload also accounts for the asynchronous memory writes. One of the main contributions is to specialize the memory read workload regarding the L2 WBs as it enables us to capture the worst-case interference contrary to previous works (e.g., [21]). Fig. 5 describes the generated events from one memory request of each workload. The next section gives an in-depth overview of these workloads.

Parameters. We have one physical core enabled.

7.1.1. Memory read workloads

A read request in the worst case corresponds to a memory read operation. Since we consider this worst-case in our workload, read request and memory read operation are interchangeable terms.

However, we identify two different L2 traffic patterns for this worst-case due to the LLC exclusivity in the processor.

In the exclusive architecture, once the memory read workload fills the L2 cache space, L2 WBs to the LLC occur whenever new memory is read. Our two memory read workloads are represented in Fig. 5 (a) and (b), respectively. We call the first memory read workload an "Exclusive DRAM Bomb" taking inspiration from [21] (which considers only the second memory read workload).

We include the two L2 traffic patterns as a factor motivating this choice with the following example.

In Fig. 6 and Fig. 7, we fetch 2 MB of dynamic memory in several blocks of 64 KB when MBA is disabled (0-delay) and enabled with 90-delay, respectively. Initially, the private caches are empty. While the upper plot of Fig. 7 represents the latency (CPU cycles) to fetch each block, the lower plot shows the respective L2 WBs for each fetched block. The number of L2 WBs grows as the L2 cache fills its capacity (L2 size is 1 MB). Initially, no L2 WBs are observed (when L2 capacity is available); subsequently, when L2 becomes full, each memory read

Table 5
Design of experiments: single-core analysis.

Metric	Description
Memory fetching latency	$L_{k;d},\ k \in K, d \in D$
Factors	Values
MBA delay	$d \in D, D = \{10; 40; 50; 60; 70; 80; 90\}$
Workload	$k \in K, K = \{R^I; R^{II}; W^{II}\}$ R^I : 8192 CAS R, no L2 WBs, no LLC WBs} {R^{II}: 8192 CAS R, with L2 WBs, no LLC WBs} { W^{II}: { 8192 CAS R, with L2 WBs, with LLC WBs }
Parameters	setting
No. cores	1

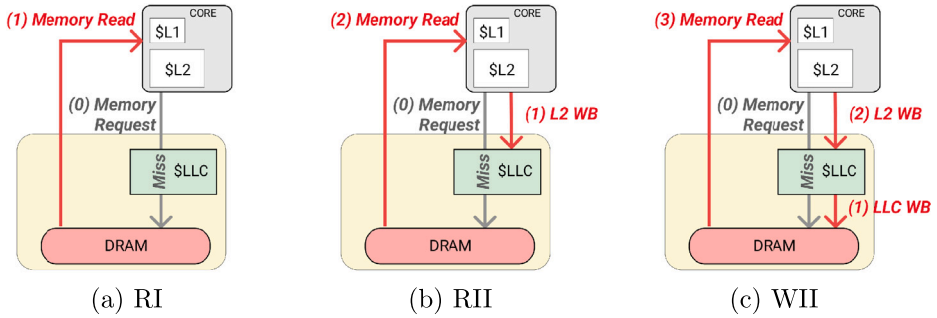


Fig. 5. Synthetic workloads designed to assess Intel MBA.

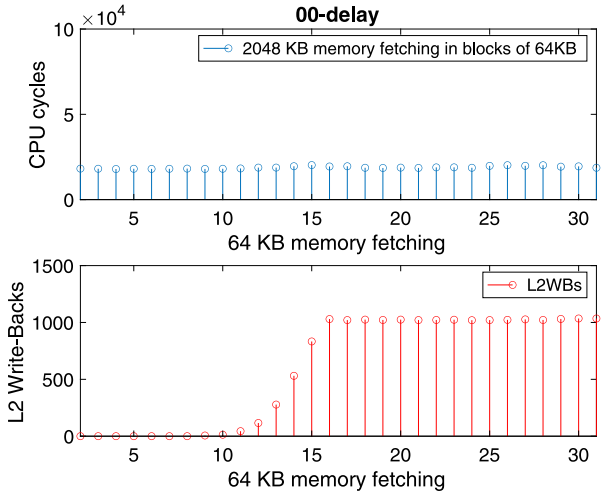


Fig. 6. 2048 KB memory fetching in several blocks of 64 KB when MBA is disabled (0-delay).

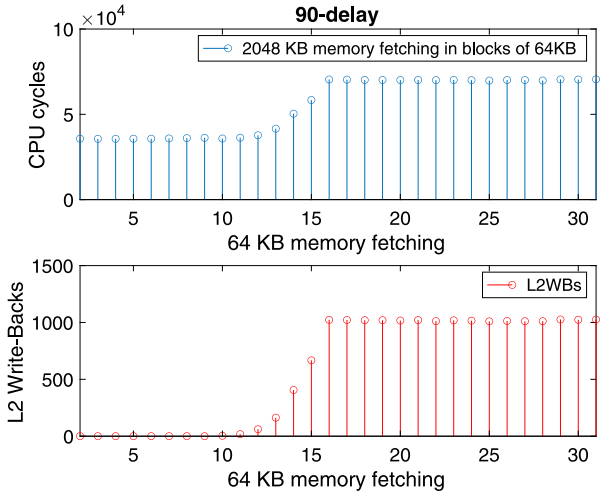


Fig. 7. 2048 KB memory fetching in several blocks of 64 KB when MBA 90-delay is enabled.

will correspond to one L2 WB (1024 memory reads of 64 bytes generate 1024 L2 WBs).

Looking at the figures, we have two main observations. (i) Without throttling (0-delay), the memory load latency does not change significantly when the L2 WBs are present (Fig. 6).[8] However, (ii) the latency changes when MBA is enabled (Fig. 7) on the two traffic patterns. From the results on CPU cycles, it is evident that MBA delays the L2 WBs requests, doubling the latency of read operations when WBs are present. This observation can be explained by the MBA controller introducing a delay toward all outgoing requests on the high-speed interconnect including reads and writes to the LLC.

The second observation motivates us to consider the L2 traffic pattern as a factor. The R^I and R^{II} in Table 5 represent the read workload respectively in the first (without L2 WBs) and the second operational condition (with L2 WBs). R^I is the Exclusive DRAM Bomb workload.

In particular, the read workload is composed of 8192 read requests where each request is a LLC miss ($readReqs = LLCmisses = CASReads = 8192$) and hence a memory read operation (CAS R is the command issued on the DRAM channel). We chose 8192 requests (meaning 512 KB memory fetches because each request retrieves a cache line size of 64 bytes) since it assures us to not have L2 WBs in R^I (L2 WBs = 0), given the L2 cache size of 1MB. Such behavior is also visible from Fig. 6, where after 8 fetches of 64 KB each, the number of L2 WBs is still 0. Conversely, in R^{II}, the total L2 WBs must be equal to the total memory requests (L2 WBs = 8192). This condition is

[8] Note that our memory read workloads are designed to not cause LLC WBs.

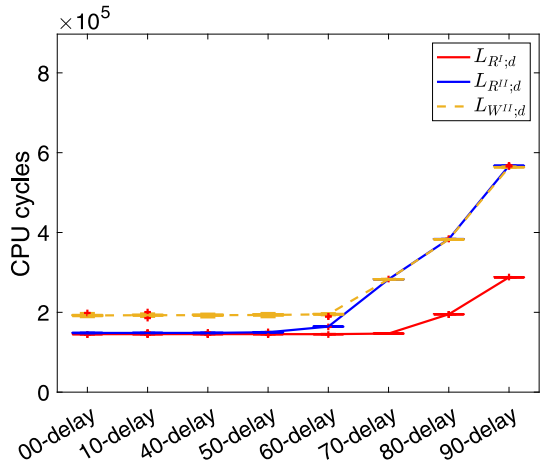


Fig. 8. Latencies for the different workloads and interpolation of the respective mean latencies.

achieved by performing a pre-read workload to populate the L2 cache. Note that, considering the results in Fig. 6 and Fig. 7, R^I is the worst case workload (it has less limitation) since the frequency (latency) of memory reads is not reduced (delayed) by concurrent L2 WBs. As a side note, looking at Fig. 7, we can observe that MBA keeps the same limitation for each of the 1024 memory reads. Such a workload in software-based approaches (e.g., [25]) would require setting an infeasible regulation period of 10 microseconds, thereby proving the better granularity offered by hardware-based MBA approach.

7.1.2. Memory write workload

We have memory writes in a WB policy when a dirty LLC cache line is evicted. Hence, since our write workload reproduces the worst-case, every workload request causes one memory read, and one memory write operation due to the cache line eviction and substitution. Fig. 5 (c) represents the events generated from each memory request of this workload.

The W^{II} in Table 5 represents this kind of workload. In particular, the W^{II} workload is composed of 8192 requests where each request is the cause of one memory read, and one memory write ($writeReqs = LLCmisses = CASReads = CASWrites = 8192$). Only the second operational condition (II) is possible for the write workload since an LLC WB can only result from an L2 WB.

7.2. Experiment observations

Below, we show our measurements for the three workloads R^I, R^{II}, W^{II}. $L_{k;d}$ is the execution latency of the workload k when the d MBA delay is active. Fig. 8 shows the $L_{k;d}$ boxplots for each experiment of Table 5 and a mean interpolation for the observations of the same workload and different MBA delays. Observing the boxplot width, the latency variability is the same (or low) when the limitation is enabled or disabled (0-delay). Hence, MBA does not seem to be a new source of variability. The main source of variability comes from the periodic DRAM refresh cycles.

7.3. Discussion

Fig. 8 shows two effects clearly. (i) The limitations do not intensely impact the relative 0-delay observations until the 70-delay in our setting. (ii) The boxplots of workload R^{II} and W^{II} overlap for high delays, meaning a similar behavior due to L2 WBs.

To better interpret the results from Fig. 8 for the 70, 80, and 90 delays, we prefer to go from the latency to the bandwidth domain. If T is the minimum execution latency $min(L_{R^I;0})$ to synchronously fetch 8192 cache lines (our chosen reference, as explained earlier), we calculate the achievable bandwidth (memory operations in T) of a generic configuration $k;d$, where k is the workload, and d is the delay, as:

$$BW_{k;d} = \frac{T * 8192}{L_{k;d}};$$

We report in Table 6 the maximum observed bandwidth for delays from 60 to 90. We evaluate also the maximum achievable bandwidth as a percentage of synchronous bandwidth

$$max(BW_{k;d})\% = \frac{max(BW_{k;d}) * 100}{8192}$$

We are interested in showing these percentages because in our work we intend to allocate portions of the synchronous memory bandwidth to each physical core.

Note that the synchronous memory bandwidth could be lower than the achievable memory bandwidth since the latter also considers memory writes (i.e., workload W^{II} achieves 155% of synchronous bandwidth with the less restrictive 60-delay limitation). Indeed, in the worst case of our write workload, each write request causes one memory read and one memory write operation, due to cache eviction. The "2x" in Table 6 counts this double access regarding memory writes.

We observe the following from the results in Table 6:

- $$max(BW_{R^I;d}) > max(BW_{R^{II};d}) \quad (1)$$

  Hence, the less restrictive limitation for a memory read workload is when there are no L2 WBs, confirming what we anticipated in the previous section.

- $$max(BW_{R^I;70:90}) \approx max(BW_{W^{II};70:90}) \quad (2)$$

  That is, the maximum memory accesses in the 1st and the 2nd operational condition with writes is approximately the same for 70, 80, and 90 delays.

- $$max(BW_{R^{II};70:90}) \approx \frac{max(BW_{W^{II};70:90})}{2} \quad (3)$$

  The memory read requests in the 2nd operational condition for 70,80 and 90 delays are halved due to L2 WBs. This regulation choice is pessimistic because L2 WBs do not always cause LLC WBs, and therefore write to memory.

These results confirm that when testing the memory bandwidth limitation for read operations, it is necessary to run measurements without L2 WBs (the Exclusive DRAM Bomb) capturing the most permissive limitation and hence the worst-case.

In addition, we can respond to **RQ3**: we discovered that although some delays do not contribute to the indirect memory bandwidth limitation in our setting, the delays 70, 80, and 90 have interesting behavior. Indeed, they limit and conserve their limitation even if there are memory writes. In particular, the 80 and 90 delays limit the achievable memory bandwidth to 75% and 50% of the maximum synchronous achievable memory bandwidth, respectively. In contrast, the 70 delay only avoids the overload of asynchronous requests due to L2 WBs, halving the synchronous bandwidth in the second operational condition.

These are important aspects to consider when dimensioning a system with MBA to assure a given memory bandwidth to a critical core.

Thus, in the multicore analysis performed in the next section, we evaluate the assured bandwidth when MBA limits the interfering workloads R^I only with 80 and 90 delay values, covering the significant cases of synchronous bandwidth allocation.

Table 6
The maximum achievable bandwidth (memory operations in T) and the maximum achievable bandwidth as a percentage of synchronous memory bandwidth.

Metrics	60-delay	70-delay	80-delay	90-delay
$max(BW_{R^I;d})$	8192	8079	6080	4105
$max(BW_{R^I;d})\%$	100%	98%	74%	50%
$max(BW_{R^{II};d})$	7237	4183	3080	2084
$max(BW_{R^{II};d})\%$	88%	51%	38%	25%
$max(BW_{W^{II};d})$	2 × 6376	2 × 4183	2 × 3091	2 × 2097
$max(BW_{W^{II};d})\%$	155%	101%	75%	51%

8. Assessing regulation support: Multicore analysis

In the previous section, we provide measures of the memory access limitation when MBA delay and workload are fixed. In this section, we explore the assured memory fetching latency for a critical core when MBA limits the accesses from the other cores, i.e., **RQ4**.

8.1. Experiment setup

Table 7 describes the design of the experiments.

Metric. The multicore analysis measures the memory fetching latency $L_{k';(d_1,d_2,d_3)}$ of a critical core when three interfering cores are delayed through $\{d_1, d_2, d_3\} \in D'$ MBA values. D' is the set of significant delay values identified in the single-core analysis.

Factors. The range of delays considered for the interfering cores ranges from 10 to 90; however, we will only consider the significant ones, i.e., D', depending on the result of the single-core analysis. The 80 and 90 delays reduce the achievable bandwidth by 75 and 50 percent of the achievable synchronous bandwidth, respectively. In Table 8, we associate an interference degree id to each experiment combination considering the limitation calculated in the single-core analysis.[9] An interference degree represents the synchronous memory accesses that we would execute without interference. If that load equals the number of requests performed by a single core with a maximum rate, then the interference degree is equivalent to one. The interference degree considers only the limitation of the single-core analysis and not the observed interference. Hence, for instance, the third combination of Table 8, where two cores interfere with 80-delays (75% of synch. bandwidth) and one core with 90-delay (50% of synch. bandwidth), has the same interference degree of two cores that interfere without limitation with the critical core ($75\% + 75\% + 50\% = 200\%$).

Parameters. We deploy a quad-core system where one core is critical. In particular, we consider a specific case where the critical core fetches its memory without MBA limitation, emulating a critical task that fills its private caches. In contrast, MBA limits the other three cores that perform the most pessimistic workload k' under the limitation resulting from the single-core analysis. The most pessimistic workloads are R^I and W^{II} under the significant delays D'. We choose to consider the R^I as an interfering workload because it is easier to reproduce it in a multicore scenario. In the experiments, we deploy a quad-core system where all four cores start with empty private caches. In this way, the interfering cores and critical core reproduce the R^I workload. In particular, the critical task fetches 512 KB, while the interfering cores fetch 768 KB, to ensure they last longer than the critical core.

8.2. Experiment observations

In Fig. 9, we report the boxplots of our critical latency observations for each combination of interference degrees in Table 8. We interpolate the 95 percentile. As expected, the lower the interference degree (achieved with high MBA delays) the lower the impact on the latency of the critical workload.

[9] Note that we are assuming the combinations are position-independent (i.e., The results obtained from limiting memory accesses from core 1 should be similar to the results obtained by limiting core 2 instead).

Table 7
Design of experiments: multicore analysis.

Metric	Description
Memory Fetching Latency	$L_{k';(d_1,d_2,d_3)}$
Factors	**Values**
MBA delay	$d_i \in D',\ D' = \{80; 90\}$
Parameters	**Setting**
No. crit. cores	1
Crit. workload	R^I
No. int. cores	3
Int. workload	R^I

Table 8
Critical core latency variability when the three interfering cores are throttled by 80 or 90 delay.

#80d	#90d	Interference degree id calculation	Resulting id
0	3	$(0 * (3/4) + 3 * (1/2))$	1.5
1	2	$(1 * (3/4) + 2 * (1/2))$	1.75
2	1	$(2 * (3/4) + 1 * (1/2))$	2
3	0	$(3 * (3/4) + 0 * (1/2))$	2.25

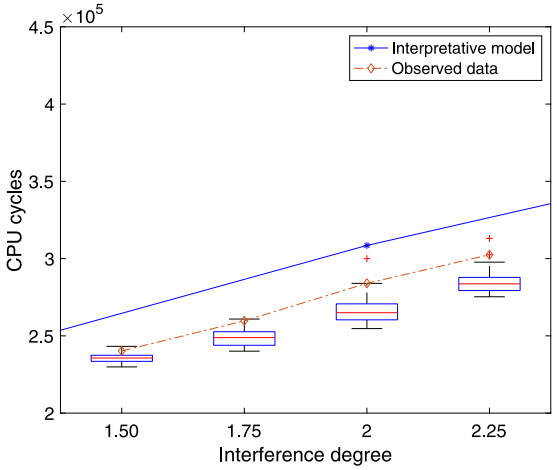


Fig. 9. Comparison between the interpretative model and the observed latencies.

8.3. Discussion

To better interpret this result, we also report the interpolation of the 95-percentile of the critical core latency when having exactly 1, 2, or 3 interfering cores with 0-delay, i.e., no MBA limitation (blue line of Fig. 9).

Comparing the two 95-percentile interpolations, we can see how the MBA is effective at limiting interference. For instance, when the interference degree is set to 2 (with 1 interfering core with 90-delay and 2 interfering cores with 80-delay), MBA is able to reduce the interference of 3 cores to at most the one we would have with 2 interfering cores running without limitation. The blue line can thus be seen as the worst-case memory access latency we can achieve when varying the interference degree, which is useful to establish predictable behavior of the critical workload when executing in a cloud. From a practical point of view, this worst-case line is also easily achievable by running interfering workloads with an increasing number of cores with no MBA.

Hence, our experiments show that MBA can provide several safety upper bounds for the memory fetching phase of a critical task in a mixed-criticality cloud infrastructure responding to **RQ4**. Such bounds

can be used in the WCET analysis of real-time workloads without requiring invasive task code instrumentation or execution interventions.

Contrarily to the Execution model, Resource Limitation through MBA does not require modifying the code of the non-critical tasks. In addition, it preserves private resource utilization limiting only the accesses to the shared context (contrarily to software solutions such as Memguard [25] which propose halting the core execution).

9. Conclusions and future work

Detection and regulation of memory access interference requires a monitoring metric to detect the degree of interference and a hardware feature to enforce the regulation. We infer the interference from the number of co-accessing cores. We identified and evaluated the queue occupancy to detect the number of co-accessing cores. At the same time, we evaluated Intel's Memory Bandwidth Allocation (MBA) for interference regulation.

We discovered that Read Pending Queue (RPQ) occupancy could be adopted to detect the number of co-accessing cores. The results are promising: at most, one-fifth of the regulation period is enough for detecting when the memory throughput is wholly consumed. In particular, only detecting more than two cores requires one-fifth of the regulation period, while detecting more than three, four, and five cores requires one-sixth of the regulation period. Ideally and theoretically, we expect a detection time equal to zero when the memory throughput is fully consumed, e.g., we expect that some queue positions are never achievable when the number of the co-accessing cores is limited. However, the ideal case does not consider real sources of interference, such as the DRAM refresh cycles where the memory stops to service requests. MBA is an interesting and reliable hardware feature for Resource limitation in real-time systems. It can be used to regulate access contention by limiting the arrival rate from the physical cores.

We provided a systematic approach to estimate the indirect limitation of MBA using three main workloads: memory reads with no L2 WBs (the Exclusive DRAM Bomb), memory reads with L2 WBs, and memory reads with L2 WBs and memory write. Experiments on a 2nd generation Intel Xeon processor revealed that only Intel's 70, 80, and 90 delay values lead to significant limitations in our setting. We discovered that the worst-case is reproducible for these delays by running our Exclusive DRAM Bomb. In addition, such limitations remain valid also when there are memory writes, that are not considered by state-of-the-art software-based solutions such as Memguard [25]. Our evaluation in the multi-core analysis showed that MBA is able to provide memory bandwidth isolation for a critical task under heavy memory-intensive workloads on multi-core platforms. In addition, MBA enables short regulation periods that would be difficult to achieve via software-based regulation methods. The high resolution is fundamental to reacting quickly when interference is detected. Finally, MBA hardware limitation does not require stopping the execution of interfering cores to prevent memory access, improving overall efficiency.

Although we performed experiments for a specific processor model, we believe the proposed method is general enough to be applicable to processor models with the latest versions of MBA technology. If newer versions of the MBA make more significant delay values available for the limitation, our current approach should be able to utilize them.

As future work, we plan to design an efficient memory bandwidth regulation algorithm following the detection/regulation approach presented in this work. In addition, we aim to leverage our new detection mechanism to scale memory voltage/frequency to improve the current memory power management solutions.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] M. Cinque, D. Cotroneo, L. De Simone, S. Rosiello, Virtualizing mixed-criticality systems: A survey on industrial trends and issues, Future Gener. Comput. Syst. 129 (2022) 315–330, http://dx.doi.org/10.1016/j.future.2021.12.002, URL https://www.sciencedirect.com/science/article/pii/S0167739X21004787.

[2] Online, Digitale Schiene Deutschland, 2022, URL https://digitale-schiene-deutschland.de/en. (Last Accessed:12/22).

[3] Thales, SYSGO, Fraunhofer IESE, University of Rostock, ESE, D. Netz, Research report - SIL4 cloud, 2022, URL https://digitale-schiene-deutschland.de/Downloads/Report%20-%20SIL4%20Cloud.pdf.(Last Accessed:12/22).

[4] G. Gala, G. Fohler, P. Tummeltshammer, S. Resch, R. Hametner, RT-Cloud: Virtualization technologies and cloud computing for railway use-case, in: 24th IEEE International Symposium on Real-Time Distributed Computing, IEEE ISORC, IEEE, 2021.

[5] D. Cotroneo, L. De Simone, R. Natella, On temporal isolation assessment in virtualized railway signaling as a service systems, 2022, http://dx.doi.org/10.48550/ARXIV.2208.14109, arXiv, URL https://arxiv.org/abs/2208.14109.

[6] Thales, TransVital, 2022, URL https://www.thalesgroup.com/en/markets/transport/main-line-rail/transvitaltm-secured-platform-sil-4-applications. (Last Accessed:12/22).

[7] Siemens, Infrastructure in the Cloud, 2022, URL https://www.mobility.siemens.com/global/en/portfolio/rail/automation/infrastructure-in-the-cloud.html. (Last Accessed:12/22).

[8] R. Hametner, P. Tummeltshammer, S. Resch, W. Wernhar, Cloud architecture for SIL4 railway applications, March 2022, Thales Austria GmbH, Signal + Draht - Signalling & Datacommunication Magazine, Eurailpress Archiv - DVV Media Group GmbH.

[9] G. Shirasat, ARM, The Cloud-Native Approach To the Software Defined Car, 2022, URL https://community.arm.com/arm-community-blogs/b/embedded-blog/posts/cloud-native-approach-to-the-software-defined-car?_ga=2.84349662.63983164.1633923986-188627514.1623282169&_gac=1.188151258.1631741153.Cj0KCQjws4aKBhDPARIsAIWH0JWzFX7JhWfY12ecutW_Gaiy3HwXQ1QWT1HbMuvrAnTdtsTSdk57dzAaAq7DEALw_wcB. (Last Accessed:12/22).

[10] Online, Scalable Open Architecture for Embedded Edge (SOAFEE), 2022, URL https://www.soafee.io/. (Last Accessed:12/22).

[11] J. Ecker, O. Angelsmark, A. Sefidcon, What Is the Real-Time Cloud and how Do We Get There?, 2022, URL https://sigbed.org/2021/02/26/what-is-the-real-time-cloud-and-how-do-we-get-there/. (Last Accessed:12/22).

[12] S. Xi, M. Xu, C. Lu, L.T. Phan, C. Gill, O. Sokolsky, I. Lee, Real-time multi-core virtual machine scheduling in Xen, in: Proc. EMSOFT, IEEE, 2014, pp. 1–10.

[13] S. A.G., Jailhouse hypervisor source code, URL https://github.com/siemens/jailhouse.

[14] J. Reineke, B. Wachter, S. Thesing, R. Wilhelm, I. Polian, J. Eisinger, B. Becker, A definition and classification of timing anomalies, in: WCET, 2006.

[15] T. Lundqvist, P. Stenström, Timing anomalies in dynamically scheduled microprocessors, in: Proceedings 20th IEEE Real-Time Systems Symposium (Cat. No.99CB37054), 1999, pp. 12–21.

[16] J. Nowotsch, M. Paulitsch, Quality of service capabilities for hard real-time applications on multi-core processors, in: Proceedings of the 21st International Conference on Real-Time Networks and Systems, RTNS '13, Association for Computing Machinery, New York, NY, USA, 2013, pp. 151–160, http://dx.doi.org/10.1145/2516821.2516826.

[17] R. Pellizzoni, A. Schranzhofer, J.-J. Chen, M. Caccamo, L. Thiele, Worst case delay analysis for memory interference in multicore systems, in: 2010 Design, Automation & Test in Europe Conference & Exhibition, DATE 2010, 2010, pp. 741–746.

[18] N. Suzuki, H. Kim, D.d. Niz, B. Andersson, L. Wrage, M. Klein, R. Rajkumar, Coordinated bank and cache coloring for temporal protection of memory accesses, in: 2013 IEEE 16th International Conference on Computational Science and Engineering, 2013, pp. 685–692, http://dx.doi.org/10.1109/CSE.2013.106.

[19] B.C. Ward, J.L. Herman, C.J. Kenna, J.H. Anderson, Outstanding paper award: Making shared caches more predictable on multicore platforms, in: 2013 25th Euromicro Conference on Real-Time Systems, 2013, pp. 157–167, http://dx.doi.org/10.1109/ECRTS.2013.26.

[20] A. Farshin, A. Roozbeh, G.Q. Maguire, D. Kostic, Make the most out of last level cache in intel processors, in: Proceedings of the Fourteenth EuroSys Conference 2019, 2019.

[21] P. Sohal, M.G. Bechtel, R. Mancuso, H. Yun, O. Krieger, A closer look at intel Resource Director Technology (RDT), in: Proceedings of the 30th International Conference on Real-Time Networks and Systems, 2022.

[22] M. Xu, L.T.X. Phan, H.-Y. Choi, Y. Lin, H. Li, C. Lu, I. Lee, Holistic resource allocation for multicore real-time systems, in: 2019 IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS, 2019, pp. 345–356, http://dx.doi.org/10.1109/RTAS.2019.00036.

[23] R. Gifford, N. Gandhi, L.T.X. Phan, A. Haeberlen, DNA: Dynamic resource allocation for soft real-time multicore systems, in: 2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium, RTAS, 2021, pp. 196–209, http://dx.doi.org/10.1109/RTAS52030.2021.00024.

[24] L. Fenn, R.D. Hawkins, P. Williams, T.P. Kelly, M.G. Banner, Y. Oakshott, The who, where, how, why and when of modular and incremental certification, in: 2007 2nd Institution of Engineering and Technology International Conference on System Safety, IET, 2007, pp. 135–140.

[25] H. Yun, G. Yao, R. Pellizzoni, M. Caccamo, L. Sha, MemGuard: Memory bandwidth reservation system for efficient performance isolation in multi-core platforms, in: 2013 IEEE 19th Real-Time and Embedded Technology and Applications Symposium, RTAS, 2013, pp. 55–64.

[26] Intel, IntelResource Director Technology (Intel RDT) On 2nd Generation Intel Xeon Scalable Processor Reference Manual, April 2019, URL https://software.intel.com/content/dam/develop/external/us/en/documents/180115-intel-rdtcascadelake-serverreferencemanual-806717.pdf.

[27] G. Farina, G. Gala, M. Cinque, G. Fohler, Assessing intel's memory bandwidth allocation for resource limitation in real-time systems, 2022.

[28] S.A. Edwards, E.A. Lee, The case for the precision timed (PRET) machine, in: 2007 44th ACM/IEEE Design Automation Conference, 2007, pp. 264–265.

[29] J. Reineke, I. Liu, H.D. Patel, S. Kim, E.A. Lee, PRET DRAM controller: Bank privatization for predictability and temporal isolation, in: 2011 Proceedings of the Ninth IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis, CODES+ISSS, 2011, pp. 99–108, http://dx.doi.org/10.1145/2039370.2039388.

[30] S. Chattopadhyay, L.K. Chong, A. Roychoudhury, T. Kelter, P. Marwedel, H. Falk, A unified WCET analysis framework for multi-core platforms, in: IEEE Real-Time and Embedded Technology and Applications Symposium, 2012.

[31] Intel, Intel 64 and IA-32 Architectures Software Developer's Manual, 2020.

[32] A. Schranzhofer, R. Pellizzoni, J.-J. Chen, L. Thiele, M. Caccamo, Timing analysis for resource access interference on adaptive resource arbiters, in: 2011 17th IEEE Real-Time and Embedded Technology and Applications Symposium, 2011, pp. 213–222, http://dx.doi.org/10.1109/RTAS.2011.28.

[33] P. Houdek, M. Sojka, Z. Hanzálek, Towards predictable execution model on ARM-based heterogeneous platforms, in: 2017 IEEE 26th International Symposium on Industrial Electronics, ISIE, 2017, pp. 1297–1302, http://dx.doi.org/10.1109/ISIE.2017.8001432.

[34] R. Pellizzoni, E. Betti, S. Bak, G. Yao, J. Criswell, M. Caccamo, R. Kegley, A predictable execution model for COTS-based embedded systems, in: 2011 17th IEEE Real-Time and Embedded Technology and Applications Symposium, 2011, pp. 269–279.

[35] G. Yao, R. Pellizzoni, S. Bak, E. Betti, M. Caccamo, Memory-centric scheduling for multicore hard real-time systems, Real-Time Syst. 48 (6) (2012) 681–715.

[36] F. Boniol, H. Cassé, É. Noulard, C. Pagetti, Deterministic execution model on COTS hardware, in: ARCS, 2012.

[37] A. Biondi, M. Di Natale, Achieving predictable multicore execution of automotive applications using the LET paradigm, in: 2018 IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS, 2018, pp. 240–250, http://dx.doi.org/10.1109/RTAS.2018.00032.

[38] C.M. Kirsch, A. Sokolova, The logical execution time paradigm, in: Advances in Real-Time Systems, Springer Berlin Heidelberg, 2012.

[39] T. Kelter, T. Harde, P. Marwedel, H. Falk, Evaluation of resource arbitration methods for multi-core real-time systems, in: 13th International Workshop on Worst-Case Execution Time Analysis, Dagstuhl, Germany, 2013, http://dx.doi.org/10.4230/OASIcs.WCET.2013.1, URL http://drops.dagstuhl.de/opus/volltexte/2013/4117.

[40] F. Bellosa, The Third Dimension of Scheduling, 1999.

[41] H. Yun, G. Yao, R. Pellizzoni, M. Caccamo, L. Sha, Memory access control in multiprocessor for real-time systems with mixed criticality, in: 2012 24th Euromicro Conference on Real-Time Systems, 2012, pp. 299–308.

[42] A. Agrawal, G. Fohler, J. Nowotsch, S. Uhrig, M. Paulitsch, Poster abstract: Slot-level time-triggered scheduling on COTS multicore platform with resource contentions, in: 2016 IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS, 2016, http://dx.doi.org/10.1109/RTAS.2016.7461353.

[43] G. Fohler, G. Gala, D. Gracia Pérez, C. Pagetti, Evaluation of DREAMS resource management solutions on a mixed-critical demonstrator, in: ERTS 2018, in: 9th European Congress on Embedded Real Time Software and Systems (ERTS 2018), Toulouse, France, 2018, URL https://hal.archives-ouvertes.fr/hal-01700860.

[44] G.J. Gala, Resource Management for Real-time and Mixed-Critical Systems (Ph.D. thesis), Technische Universität Kaiserslautern, 2021.

[45] F. Bellosa, Process cruise control: Throttling memory access in a soft real-time environment, in: SOSP 1997, 1997.

[46] J. Nowotsch, M. Paulitsch, D. Bühler, H. Theiling, S. Wegener, M. Schmidt, Multi-core interference-sensitive WCET analysis leveraging runtime resource capacity enforcement, in: 2014 26th Euromicro Conference on Real-Time Systems, IEEE, 2014, pp. 109–118.

[47] Y. Zhou, D. Wentzlaff, MITTS: Memory inter-arrival time traffic shaping, in: 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture, ISCA, 2016, pp. 532–544, http://dx.doi.org/10.1109/ISCA.2016.53.

[48] M. Pagani, E. Rossi, A. Biondi, M. Marinoni, G. Lipari, G.C. Buttazzo, A bandwidth reservation mechanism for AXI-based hardware accelerators on FPGAs, in: ECRTS, 2019.

[49] F. Farshchi, Q. Huang, H. Yun, BRU: Bandwidth regulation unit for real-time multicore processors, in: 2020 IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS, 2020, pp. 364–375, http://dx.doi.org/10.1109/RTAS48715.2020.00011.

[50] P. Sohal, R. Tabish, U. Drepper, R. Mancuso, E-WarP: A system-wide framework for memory bandwidth profiling and management, in: 2020 IEEE Real-Time Systems Symposium, RTSS, 2020, pp. 345–357, http://dx.doi.org/10.1109/RTSS49844.2020.00039.

[51] H. Aghilinasab, W. Ali, H. Yun, R. Pellizzoni, Dynamic memory bandwidth allocation for real-time GPU-based SoC platforms, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 39 (11) (2020) 3348–3360, http://dx.doi.org/10.1109/TCAD.2020.3012210.

[52] A. Serrano-Cases, J.M. Reina, J. Abella, E. Mezzetti, F.J. Cazorla, Leveraging hardware QoS to control contention in the Xilinx Zynq UltraScale+ MPSoC, in: B.B. Brandenburg (Ed.), 33rd Euromicro Conference on Real-Time Systems, ECRTS 2021, in: Leibniz International Proceedings in Informatics (LIPIcs), vol. 196, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2021, pp. 3:1–3:26, http://dx.doi.org/10.4230/LIPIcs.ECRTS.2021.3, URL https://drops.dagstuhl.de/opus/volltexte/2021/13934.

[53] A. Limited, Arm Architecture Reference Manual Supplement Memory System Resource Partitioning and Monitoring (MPAM), for a-Profile Architecture, 2022.

[54] M. Zini, D. Casini, A. Biondi, Analyzing arm;s MPAM from the perspective of time predictability, IEEE Trans. Comput. (2022) 1–14, http://dx.doi.org/10.1109/TC.2022.3202720.

[55] H. Kim, D. de Niz, B. Andersson, M. Klein, O. Mutlu, R. Rajkumar, Bounding memory interference delay in COTS-based multi-core systems, in: 2014 IEEE 19th Real-Time and Embedded Technology and Applications Symposium, RTAS, 2014, pp. 145–154, http://dx.doi.org/10.1109/RTAS.2014.6925998.

[56] H. Yun, R. Pellizzon, P.K. Valsan, Parallelism-aware memory interference delay analysis for COTS multicore systems, in: 2015 27th Euromicro Conference on Real-Time Systems, 2015, pp. 184–195, http://dx.doi.org/10.1109/ECRTS.2015.24.

[57] P.K. Valsan, H. Yun, MEDUSA: A predictable and high-performance DRAM controller for multicore based embedded systems, in: 2015 IEEE 3rd International Conference on Cyber-Physical Systems, Networks, and Applications, 2015, pp. 86–93, http://dx.doi.org/10.1109/CPSNA.2015.24.

[58] H. Yun, R. Mancuso, Z. Wu, R. Pellizzoni, PALLOC: DRAM bank-aware memory allocator for performance isolation on multicore platforms, in: 2014 IEEE 19th Real-Time and Embedded Technology and Applications Symposium, RTAS, 2014, pp. 155–166.

[59] M. Masmano, I. Ripoll, A. Crespo, J. Metge, Xtratum: A hypervisor for safety critical embedded systems, in: 11th Real-Time Linux Workshop, Citeseer, 2009, pp. 263–272.

[60] R. Mancuso, R. Pellizzoni, M. Caccamo, L. Sha, H. Yun, WCET(m) estimation in multi-core systems using single core equivalence, in: 2015 27th Euromicro Conference on Real-Time Systems, 2015, pp. 174–183, http://dx.doi.org/10.1109/ECRTS.2015.23.

[61] A. Agrawal, R. Mancuso, R. Pellizzoni, G. Fohler, Analysis of dynamic memory bandwidth regulation in multi-core real-time systems, 2018, arXiv:1809.05921.

[62] M. Behnam, R. Inam, T. Nolte, M. Sjödin, Multi-core composability in the face of memory-bus contention, SIGBED Rev. 10 (3) (2013) http://dx.doi.org/10.1145/2544350.2544354.

[63] A. Agrawal, G. Fohler, J. Freitag, J. Nowotsch, S. Uhrig, M. Paulitsch, Contention-aware dynamic memory bandwidth isolation with predictability in COTS multicores: An avionics case study, in: M. Bertogna (Ed.), 29th Euromicro Conference on Real-Time Systems, ECRTS 2017, in: Leibniz International Proceedings in Informatics (LIPIcs), 76, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 2017, pp. 2:1–2:22, http://dx.doi.org/10.4230/LIPIcs.ECRTS.2017.2, URL http://drops.dagstuhl.de/opus/volltexte/2017/7174.

[64] D. Molka, R. Schöne, D. Hackenberg, W.E. Nagel, Detecting memory-boundedness with hardware performance counters, in: Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering, 2017.

[65] C. Helm, S. Akiyama, K. Taura, Reliable reverse engineering of intel DRAM addressing using performance counters, in: 2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems, MASCOTS, 2020, pp. 1–8, http://dx.doi.org/10.1109/MASCOTS50786.2020.9285962.

[66] C. Maurice, N.L. Scouarnec, C. Neumann, O. Heen, A. Francillon, Reverse engineering intel last-level cache complex addressing using performance counters, in: RAID, 2015.

[67] M.Y. Lim, A. Porterfield, R.J. Fowler, SoftPower: Fine-grain power estimations using performance counters, in: HPDC '10, 2010.

[68] É. André, R. Dulong, A. Guermouche, F. Trahay, duf: Dynamic uncore frequency scaling to reduce power consumption, Concurr. Comput.: Pract. Exper. 34 (2022).

[69] Intel, Intel Xeon processor scalable memory family uncore performance monitoring, 2020.